ESTRUCTURA DE DATOS

DANCY DAYANA GOMEZ GALINDEZ ANA SOFIA ANGEL GALAN

Docente: GORGE GAMARRA

UNIVERSIDAD REMINGTON DE CALI

Ingeniería En Sistema

Cali, Abril 2026

3. Instrucciones del Taller

1. Analizar la situación problema e identificar:

- **Tipo de operaciones más frecuentes**
- **Necesidad de acceso rápido**
- **Tipo de recorrido requerido**

RP//: La empresa necesita:

- Buscar envíos por código rápidamente (CRÍTICO)
- Insertar envíos constantemente
- Eliminar envíos
- Recorrer envíos en uno o ambos sentidos
- Soportar crecimiento de datos sin volverse lento
- También se requiere flexibilidad (insertar/eliminar)

2. Seleccionar la estructura de datos más adecuada:

- **Justificar la elección**
- **Compararla con al menos una de las otras opciones.**

✓ Justificación

Se elige una tabla hash porque:

- Permite buscar por código en tiempo constante O(1)
- Inserta y elimina rápidamente
- Escala bien cuando hay muchos envíos

La tabla hash permite hacer todo esto sin afectar el rendimiento global argumento fuerte)

Compararla con al menos una de las otras opciones

Las comparaciones teniendo en cuenta lo visto Una lista doblemente enlazada permite recorrer en ambos sentidos, Pero no resuelve el problema principal: la búsqueda rápida, aunque la lista es útil para el recorrido es muy lento.

3. Justificar la decisión utilizando notación Big O:

- **Inserción**

Tabla hash O(1)

Lista simplemente enlazada O(1) (si se inserta al inicio)

Lista doblemente enlazada O(1)

Ambas estructuras son eficientes para insertar, pero la tabla hash mantiene esa eficiencia sin afectar la búsqueda

- **Búsqueda**

Tabla hash O(1) (tiempo constante, en promedio)

Lista simplemente enlazada O(n) (debe recorrer todos los elementos)

Lista doblemente enlazada O(n)

La operación más importante del sistema es buscar envíos por código.
La tabla hash permite encontrar un envío de forma inmediata, mientras que una lista enlazada se vuelve lenta a medida que crecen los datos

- **Eliminación.**

Tabla hash O(1) promedio (si se conoce el índice)

Lista simplemente enlazada O(n) (hay que buscar primero)

Lista doblemente enlazada O(n)

En la tabla hash se elimina rápidamente, mientras que en listas enlazadas primero hay que recorrer para encontrar el elemento.

En las listas (simples y dobles) primero hay que buscar el nodo, lo que toma O(n) aunque la lista doble facilita eliminar una vez encontrado el nodo, pero el costo de búsqueda sigue siendo el problema. La tabla hash sigue siendo la más eficiente.

Operación	Tabla hash	Lista simple	Lista doble
Inserción	O(1)	O(1)	O(1)
Búsqueda	O(1)	O(n)	O(n)
Eliminación	O(1)	O(n)	O(n)

7. Preguntas de Reflexión

1. ¿Cómo cambiaría la eficiencia del sistema si se utilizara otra estructura de datos diferente a la seleccionada?

RP//: Sería más lenta, especialmente la búsqueda (pasaría de O(1) a O(n)). Teniendo en cuenta la anotación Big O,

2. ¿Qué limitaciones presenta la estructura elegida en escenarios de gran volumen de datos?

RP//: Teniendo en cuenta lo visto en clases las limitaciones de la tabla hash son que:

- Puede haber colisiones
- No mantiene orden
- Recorrido no es natural en ambos sentidos

3. ¿En qué casos reales sería más conveniente utilizar una tabla hash en lugar de listas enlazadas?

RP//: La tabla hash es más conveniente que las listas enlazadas en situaciones donde se requiere acceso rápido a los datos mediante una clave única, especialmente en sistemas en tiempo real y con grandes volúmenes de información. Esto se debe a que permite realizar búsquedas, inserciones y eliminaciones en tiempo constante, mientras que las listas enlazadas requieren recorrer todos los elementos, lo que reduce su eficiencia.

Ejemplos reales:

- Sistemas de logística (buscar un envío por código)
- Bases de datos de usuarios (buscar por documento o ID)

- Inventarios (buscar producto por código
- Aplicaciones bancarias
- Sistemas de pedidos en línea
- Seguimiento de paquetes

La tabla hash es la mejor opción porque:

- Resuelve el problema principal (búsqueda rápida)
- Es eficiente con grandes volúmenes de datos
- Búsqueda rápida por clave única